Google Summer of Code

Aryan Verma
aryan.v.0999@gmail.com @aryan.verma:open.rocket.chat /
Aryan Verma

Proposal: Embedded Chat 2026, Modernize APIs, enable Matrix federation, add pluggable AI, and make EmbeddedChat fully mobile-ready and accessible

Google Summer of Code 2026

GENERAL DETAILS

Name: Aryan Verma

University: Galgotias University

Primary Email: aryan.v.0999@gmail.com

Secondary Email: aryan-999@outlook.com

Rocket.Chat ID: @aryan.verma:open.rocket.chat

GitHub: Aryan-Verma-999

LinkedIn: Aryan Verma

Country: India

Time Zone: India (GMT+5:30)

Size of the Project: Medium (175 Hours)

MENTOR

Zishan Ahmad (@Spiral-Memory)

ABSTRACT

EmbeddedChat is Rocket.Chat's embeddable chat widget, but recent Rocket.Chat releases have introduced compatibility issues in parts of its API layer, Matrix federation support is still absent, there is no clean AI extensibility point, mobile responsiveness remains inconsistent, and accessibility support is incomplete in several areas.

This project aims to address these five gaps by restoring compatibility with current Rocket.Chat releases, completing the DDP client migration, adding Matrix federation support through a dedicated mode prop, introducing a standalone provider-agnostic AI adapter package, improving small-screen usability across the widget, and bringing the component library closer to WCAG 2.1 AA compliance.

Two of these goals are already supported by working proof-of-concepts. Matrix federation is live at chat.aryanverma.dev on a patched Rocket.Chat instance, and the AI adapter prototype already supports smart reply suggestions and thread summarization through a self-hosted Ollama setup.
Demo credentials for evaluation:
Username : gsoc
Password : Embeddedchat@2026

BENEFITS TO COMMUNITY

Rocket.Chat, founded in 2016, has served 15 million users across 150 countries. It is one of the largest chat platforms that is open source. One of its projects, which can be integrated into any website as an in-chat app, is "Embedded Chat" which originated from an idea presented by the project mentor to address the iframe issue and provide easy embedding within any website. Its first commit was on 2 June 2022, and it has since helped several Rocket.Chat users in their businesses.

Over 42% of users seek live chat support, and 69% prefer self.service chat interfaces. EmbeddedChat sits at the intersection of both, but only if it works reliably with modern RC releases, supports emerging communication standards like Matrix federation, and meets the accessibility expectations of today's users.

For Host App Developers

- **RC 8.x compatibility restored:** After this project, any developer running RC 8.x+ can embed the widget without hitting broken endpoints, ghost WebSocket sessions, or React version conflicts.
- **AI features:** The @embeddedchat/ai-adapter package lets integrators wire up any AI provider (OpenAI, Ollama, RC AI backend, or their own) without touching EmbeddedChat internals.
- **Matrix federation:** Switching between standard Rocket.Chat and Matrix-federated modes requires changing one prop value. Cross-server communication becomes trivial to offer.

For End Users

- **Mobile-first experience:** Users on 375px viewports get properly rendered attachments, a full screen emoji picker instead of a cramped popup, and better kebab menus.
- **Accessible:** Screen reader users can navigate messages, member lists, and all interactive elements with full ARIA semantics and keyboard support.

These will make EC the best choice for in-app chat, overall improving customizability, security, and provide an incredibly valuable experience for both admins and their users.

GOALS

Throughout the course of GSoC my primary focus would be on:

1. Completing the RC 8.x API modernization: migrating removed endpoints, finishing the DDP client migration, and resolving the React version mismatch across the monorepo.
2. Delivering Matrix federation support through a mode prop, backed by end-to-end work across EmbeddedChat and the required Rocket.Chat federation flow.
3. Building and publishing packages/ai-adapter, a provider-agnostic AI adapter package that allows integrators to plug in OpenAI, Ollama, Rocket.Chat AI services, or custom backends without modifying EmbeddedChat core.
4. Systematically fixing mobile viewport issues identified during 375px and small-screen testing, including emoji picker behavior, kebab menu overflow, and modal responsiveness.
5. Improving accessibility toward WCAG 2.1 AA compliance through semantic ARIA roles, keyboard navigation, focus handling, and live regions.
6. Documenting the Matrix federation setup, AI adapter interface, and mobile/accessibility improvements so future contributors can extend them more easily.

DELIVERABLES

By the end of GSoC, I plan to deliver a modernized, federation-ready, and AI-extensible version of EmbeddedChat that works reliably with current Rocket.Chat releases and meets modern accessibility and mobile standards.

1. Rocket.Chat compatibility updates for current releases, including migrated deprecated endpoints, completed DDP client migration, and resolved React version mismatch across the monorepo. **[MODERNIZATION]**
2. Matrix federation support exposed through a mode prop, including self-join federation using Rocket.Chat's native federation REST API. **[NEW]**
3. A standalone packages/ai-adapter package defining an IAIAdapter interface with reference implementations such as OpenAI, Ollama, and mock adapters. **[NEW]**
4. Mobile usability improvements across primary breakpoints, including emoji picker behavior, kebab menu overflow handling, and responsive modal/layout fixes at the UI layer. **[ENHANCEMENT]**
5. Accessibility improvements across EmbeddedChat components, including semantic ARIA roles, keyboard navigation, and live region support.**[NEW]**
6. Documentation for Matrix federation setup, AI adapter usage, and the mobile/accessibility improvements delivered during the project. **[NEW]**

EXTRAS

The following are the features that I plan to build during the GSoC period only if I complete my deliverables before the scheduled plan; otherwise, I will try to build these afterward.

1. Adding i18n support for EmbeddedChat so the widget can be more easily localized across host applications. **[NEW]**
2. Adding draft persistence for unsent messages so user input is not lost on refresh, route changes, or temporary connection drops. **[NEW]**
3. Adding automated accessibility checks in CI to catch WCAG regressions early as new UI changes are introduced. **[NEW]**
4. Expanding Storybook coverage for AI adapter integrations, federation states, and accessibility-focused interaction scenarios.**[ENHANCEMENT]**

IMPLEMENTATION DETAILS

1. Completing Rocket.Chat modernization and monorepo compatibility

The most important foundation for this project is to bring EmbeddedChat fully in line with current Rocket.Chat releases. A significant part of this work has already been started in Devansh Kansagra's PR #1009, which migrates a large portion of @embeddedchat/api and @embeddedchat/auth toward @rocket.chat/ddp-client. My plan is to build on top of that work rather than replacing it.

Proposed architecture

The modernization effort is centered around three layers:

- **@embeddedchat/api**
  This package is responsible for Rocket.Chat communication and will become the single source of truth for DDP subscriptions, REST requests, and real-time room events.
- **@embeddedchat/auth**

This package handles authentication flow and token lifecycle and must be aligned with the upgraded API layer.

- **Consumer packages**

@embeddedchat/react, @embeddedchat/ui-elements, @embeddedchat/ui-kit, and @embeddedchat/markups must all move to a single React 19 baseline so the workspace is no longer split across React 17 and React 18.

At the repository level, the monorepo must also be upgraded from the currently pinned Node 16 environment to the latest Node.js 22 version, followed by fixes to Storybook, demo apps, builds, and tests.

Implementation strategy

The work will proceed in the following order:

1. Extend Devansh's DDPSDK migration and verify all major chat flows.
2. Replace remaining fallback paths that still rely on old behavior or raw fetch.
3. Remove legacy SDK dependencies still left in the API/auth layer.
4. Unify core packages on one modern React version.
5. Upgrade the root toolchain to the latest stable Node.js version.
6. Revalidate builds, Storybook, demos, and Playwright tests.

Why this approach

This keeps the compatibility work layered correctly. Instead of applying small fixes directly in the UI, I want to stabilize the transport and runtime foundation first. Once the API/auth layer is correct, the rest of the monorepo upgrade becomes much more predictable.

```
packages > react > package.json > {} devDependencies
29    "devDependencies": {
69      "react": "^17.0.2",
```

```
packages > e2e-react > package.json > {} dependencies
15    "dependencies": {
17      "react": "^18.2.0",
```

Fig. 1: Current React version mismatch across EmbeddedChat packages

```
.nvmrc
1    v16.19.0
```

Fig. 2: Repository still pinned to Node v16.19.0

Draft **Refactor: Migrating from RocketChat SDK to DDPSDK of Rocket.Chat** #1009
All commits · devanshkansagra wants to merge 3 commits into RocketChat:develop from devanshkansagra:migrate/ddp-client · 0 / 14 viewed · View status · Submit review

packages/api/src/EmbeddedChatApi.ts · +263 -394 · Viewed

```
188  188      async connect() {
189  189        try {
190       -      await this.close(); // before connection, all previous subscriptions should be cancelled
191       -      await this.rcClient.connect({});
     190  +      await this.rcClient.connection.connect();
192  191        const token = (await this.auth.getCurrentUser())?.authToken;
193       -      await this.rcClient.resume({ token });
194       -      await this.rcClient.subscribeRoom(this.rid);
195       -      await this.rcClient.onMessage((data: any) => {
     192  +      await this.rcClient.account.loginWithToken(token);
     193  +      await this.rcClient.stream(
     194  +        "notify-room",
     195  +        `${this.rid}/user-activity`,
     196  +        (...props: [string, string[]]) => {
     197  +          const [username, activities] = props;
     198  +          this.handleTypingEvent({
     199  +            typingUser: username,
     200  +            isTyping: activities.includes("user-typing"),
     201  +          });
     202  +        }
     203  +      );
     204  +
     205  +      await this.rcClient.stream(
     206  +        "notify-room",
     207  +        `${this.rid}/deleteMessage`,
     208  +        (ddpMessage: any) => {
     209  +          const messageId = ddpMessage._id;
     210  +          this.onMessageDeleteCallbacks.map((callback) => callback(messageId));
     211  +        }
     212  +      );
     213  +
     214  +      await this.rcClient.stream("room-messages", this.rid, (data: any) => {
```

Fig. 3: Devansh's DDPSDK migration base in PR #1009

2. Adding Matrix federation support through a federation-aware EmbeddedChat mode

One of the most technically interesting parts of this project is enabling EmbeddedChat to work with Rocket.Chat's homeserver and federation capabilities. My current POC already proves that Matrix federation can work in EmbeddedChat using Rocket.Chat's native federation REST flow rather than a separate Matrix SDK or Matrix Bridge, and I plan to turn that into a maintainable, upstream-friendly implementation.

Proposed architecture

My proposed federation architecture keeps federation support inside @embeddedchat/react instead of creating a separate client. The goal is to expose federation as a first-class EmbeddedChat capability rather than a separate fork.

The architecture can be described in four parts:

1. **Federation-aware client mode**

EmbeddedChat will expose a prop-based federation mode, such as mode="federation" or a dedicated federation flag. This mode will enable federation-specific UI behavior and room handling without affecting standard EmbeddedChat installations.

2. **Federation UI layer inside @embeddedchat/react**

Federation-specific UI elements such as room badges, join indicators, unsupported-state placeholders, or room metadata components will live inside the React package. This keeps the public API simple while making federation behavior explicit in the component tree.

3. **Rocket.Chat federation REST integration through @embeddedchat/api**

Room joins, federation-aware lookups, and other required server communication will be routed through the existing API layer. This ensures that federation is integrated as part of the normal EmbeddedChat networking model rather than bolted on separately.

4. **Rocket.Chat server-side federation support**

My current POC uses a patched Rocket.Chat instance to validate these flows. That POC is useful because it proves feasibility today. For the final GSoC implementation, I want to keep the EmbeddedChat side as clean and reusable as possible, while clearly documenting any Rocket.Chat-side requirements or upstream dependencies.

Data flow

The flow I intend to support is:

1. Host app enables federation mode in EmbeddedChat.
2. EmbeddedChat initializes in federation-aware mode.
3. The API layer detects whether the room or target is federated.
4. EmbeddedChat calls Rocket.Chat's native federation REST endpoints for room access and related actions.
5. Federation-specific UI is rendered in the widget, such as a federation badge or a fallback component for unsupported flows.
6. Standard chat operations continue through the existing EmbeddedChat API/auth infrastructure.

This approach avoids a split architecture where a second Matrix-specific client would have to be maintained in parallel.

Implementation strategy

The implementation can be broken into the following steps:

1. Finalize the federation mode API exposed by EmbeddedChat.
2. Move existing federation UI work into stable reusable components.
3. Integrate federation room handling into the API layer.
4. Add detection and rendering for federated room states.
5. Add fallback UX for unsupported cases such as public room discovery.
6. Document setup and provide Storybook/demo coverage.

Why this approach

This architecture is strong because it uses Rocket.Chat's own federation direction, keeps the host-app integration simple, and avoids adding a separate Matrix SDK or duplicate room model on the client.

DEMO VIDEO OF POC

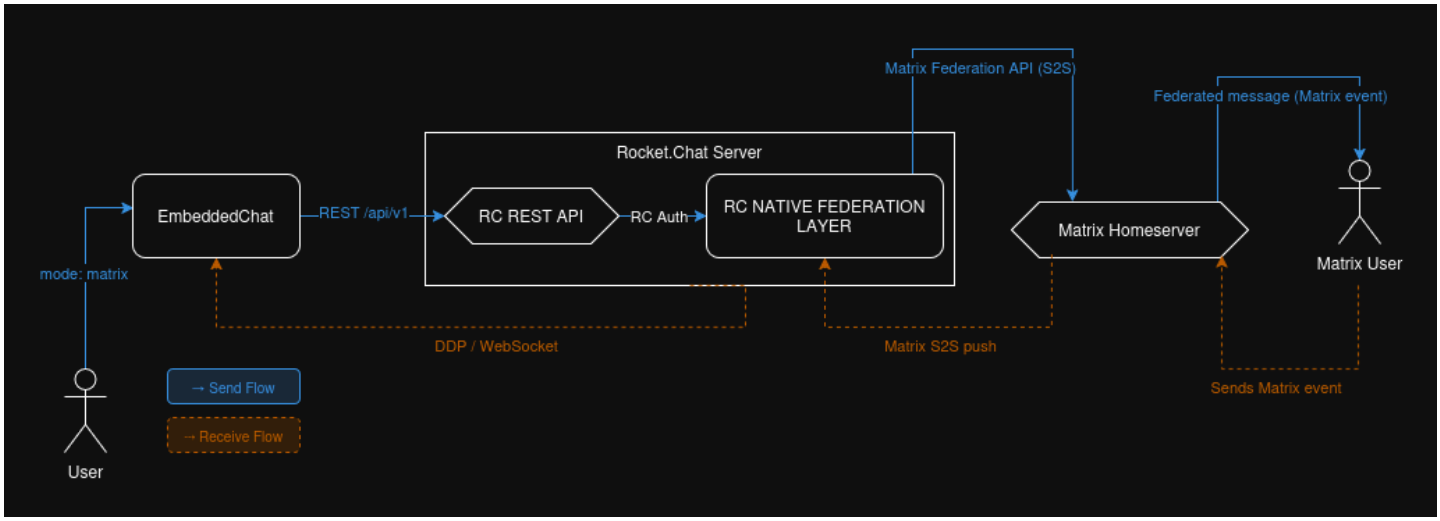


Fig. 4: EmbeddedChat Matrix federation Architecture

Code Blame 23 lines (21 loc) · 504 Bytes

```
1   import { EmbeddedChat } from '..';
2
3   export default {
4     title: 'EmbeddedChat/WithMatrix',
5     component: EmbeddedChat,
6   };
7
8   export const WithMatrix = {
9     args: {
10      host: process.env.STORYBOOK_RC_HOST || 'http://localhost:3000',
11      roomId: process.env.RC_ROOM_ID || 'GENERAL',
12      channelName: 'general',
13      anonymousMode: false,
14      toastBarPosition: 'bottom right',
15      showRoles: true,
16      enableThreads: true,
17      hideHeader: false,
18      auth: {
19        flow: 'PASSWORD',
20      },
21      dark: false,
22    },
23  };
```

Fig. 5: EmbeddedChat with Matrix federation enabled in Storybook

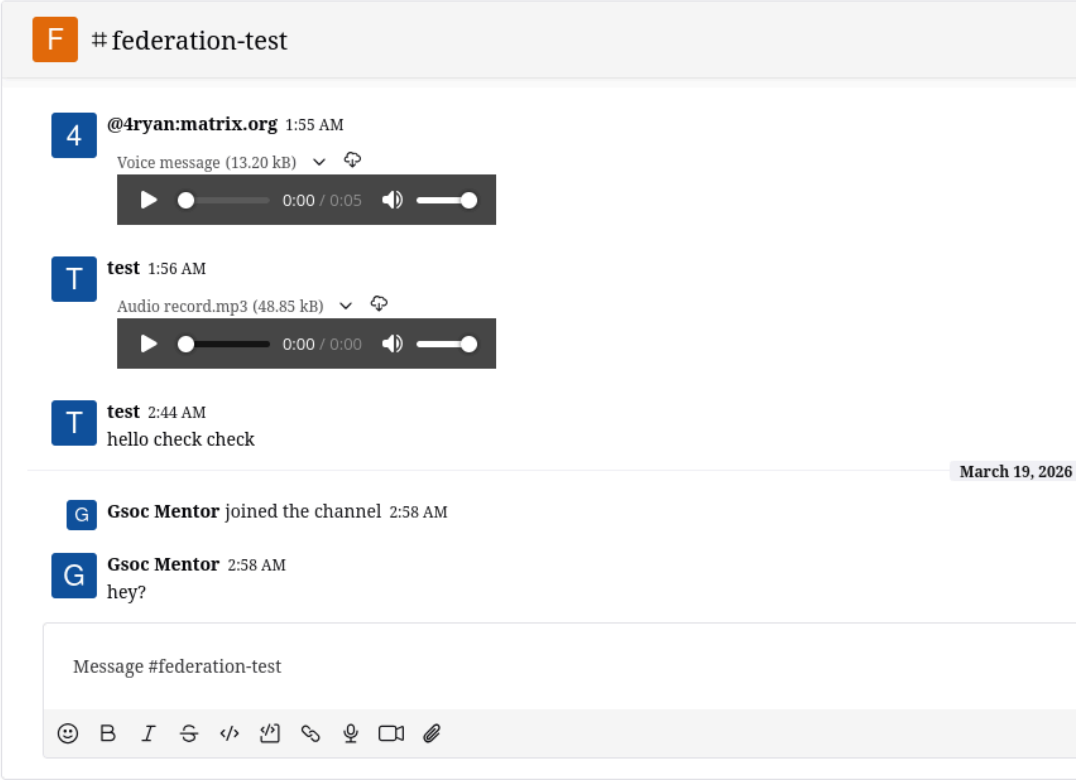


Fig 6: Live Matrix federation POC running on my demo instance

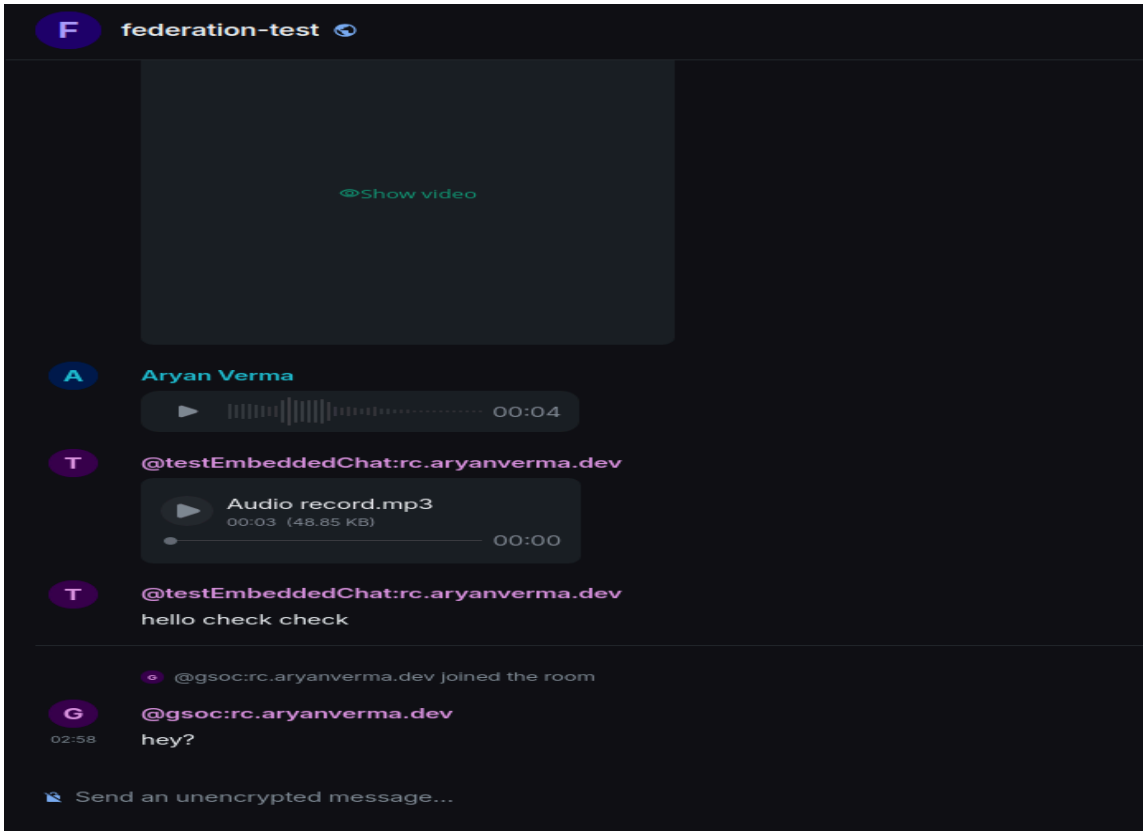
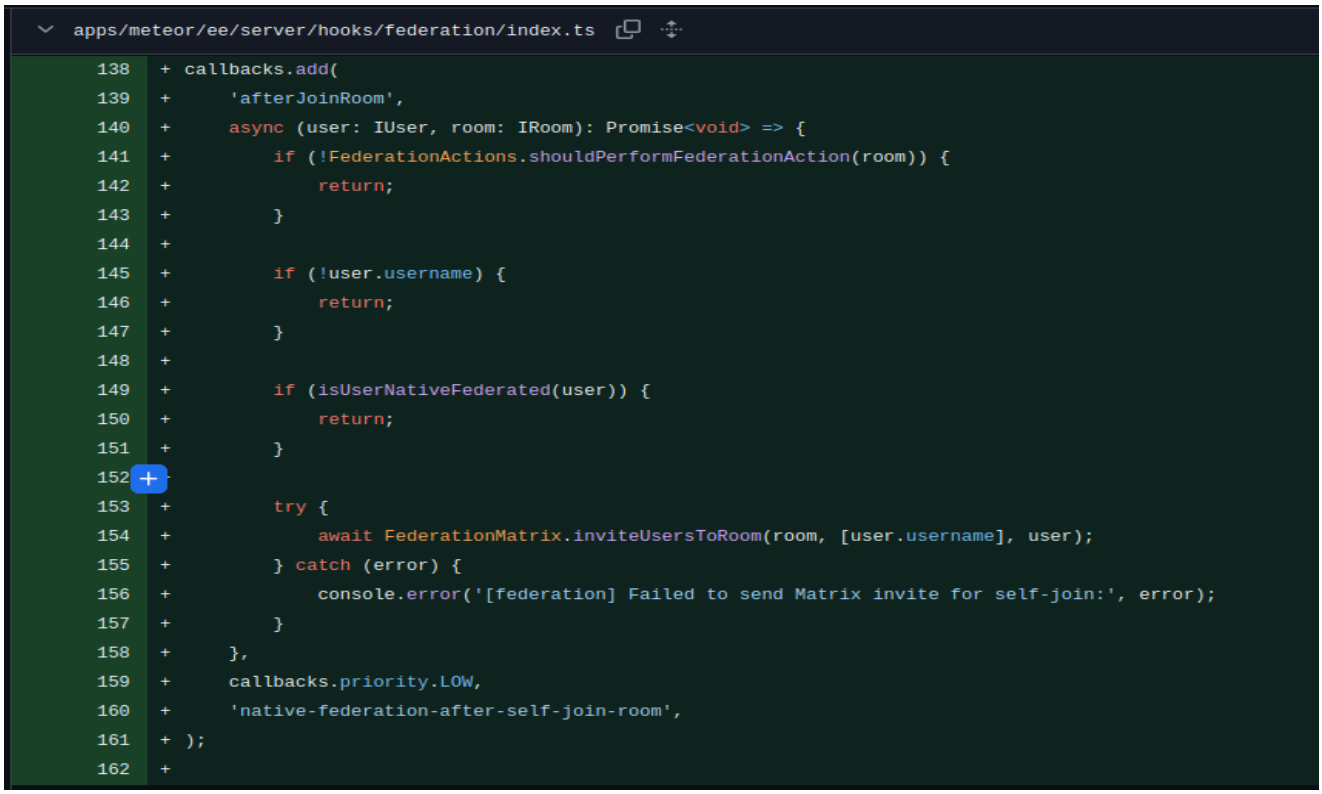


Fig. 7: Screenshot of Matrix Server Confirming user registered on rc homeserver



apps/meteor/ee/server/hooks/federation/index.ts

```
+ callbacks.add(
+     'afterJoinRoom',
+     async (user: IUser, room: IRoom): Promise<void> => {
+         if (!FederationActions.shouldPerformFederationAction(room)) {
+             return;
+         }
+
+         if (!user.username) {
+             return;
+         }
+
+         if (isUserNativeFederated(user)) {
+             return;
+         }
+
+         try {
+             await FederationMatrix.inviteUsersToRoom(room, [user.username], user);
+         } catch (error) {
+             console.error('[federation] Failed to send Matrix invite for self-join:', error);
+         }
+     },
+     callbacks.priority.LOW,
+     'native-federation-after-self-join-room',
+ );
+
```

Fig. 8: Self-Join Federation Patched in Rocket.chat Repo

3. Introducing a provider-agnostic AI adapter layer

To make AI support flexible and maintainable, I plan to keep it outside EmbeddedChat core and expose it through a standalone package, packages/ai-adapter. This ensures that EmbeddedChat does not become coupled to any single AI provider and that integrators can use local, cloud, or proprietary AI backends according to their needs.

Proposed architecture

The AI architecture is intentionally modular:

- **@embeddedchat/react**

Receives an aiAdapter prop and enables AI features only when an adapter is provided.

- **@embeddedchat/ai-adapter**

Defines the adapter contract and ships reference implementations.

- **Host application**

Owns provider configuration and credentials.

- **AI backend**

Can be OpenAI, Ollama, Rocket.Chat AI backend, Gemini, or a completely custom service.
This architecture keeps AI optional and external to the Rocket.Chat server itself.

Implementation strategy

1. Finalize and document the IAIAdapter contract.
2. Keep all provider-specific logic in the standalone package.
3. Inject the adapter into EmbeddedChat through props/context.
4. Expose UI capabilities such as smart reply suggestions and summarization.
5. Ensure zero behavior change when no adapter is provided.
6. Provide reference adapters and demo stories.

Why this approach

This design gives integrators maximum flexibility while keeping EmbeddedChat core lightweight and easier to maintain.

DEMO VIDEO OF POC

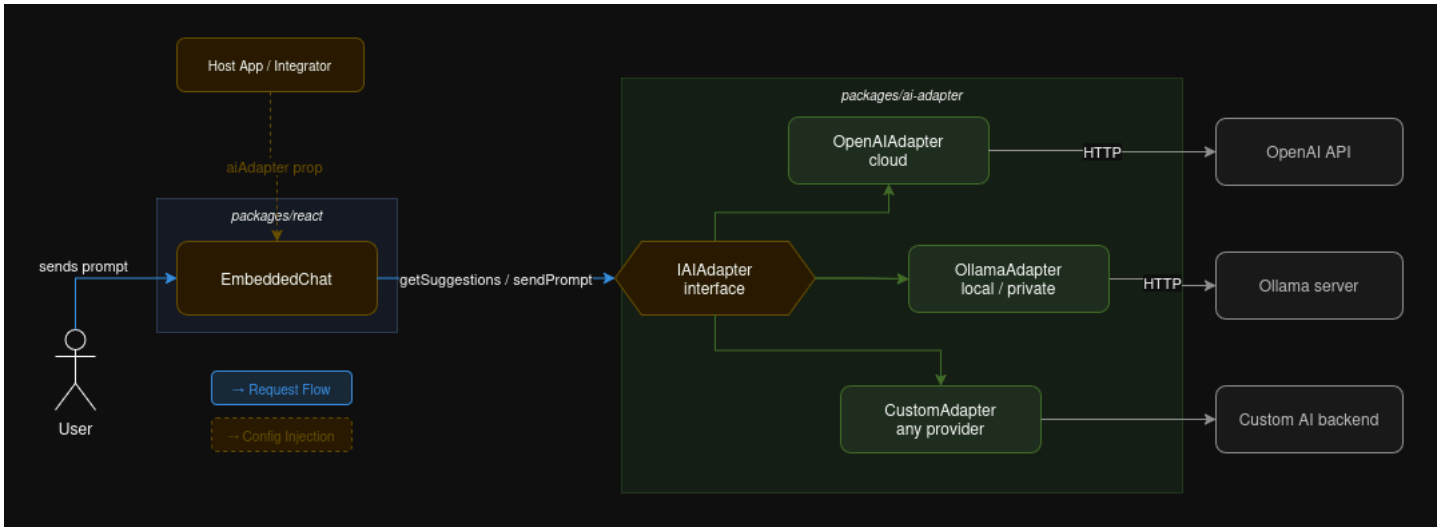


Fig. 9: Architecture of the Ai Adapter

```
packages > ai-adapter > src > TS index.ts
1  export type { IAIAdapter, AIContext, AIResponse, Message } from "./types";
2  export { BaseAIAdapter } from "./BaseAIAdapter";
3  export { OpenAIAdapter } from "./adapters/OpenAIAdapter";
4  export { OllamaAdapter } from "./adapters/OllamaAdapter";
5  export { GeminiAdapter } from "./adapters/GeminiAdapter";
6  export { MockAdapter } from "./adapters/MockAdapter";
7
```

Fig. 10: Built-in reference adapters exported by the package

```
const handleGetSuggestions = async () => {
  if (!aiAdapter || isFetchingSuggestions) return;
  setIsFetchingSuggestions(true);
  try {
    const { messages } = useMessageStore.getState();
    const suggestions = aiAdapter.getSuggestions
      ? await aiAdapter.getSuggestions(messages.slice(-10))
      : [];
    setAiSuggestions(suggestions);
  } catch (e) {
    console.error('[AI Adapter] getSuggestions failed:', e);
  } finally {
    setIsFetchingSuggestions(false);
  }
};
```

Fig. 11: Ai Suggestions Hook inside chatinput

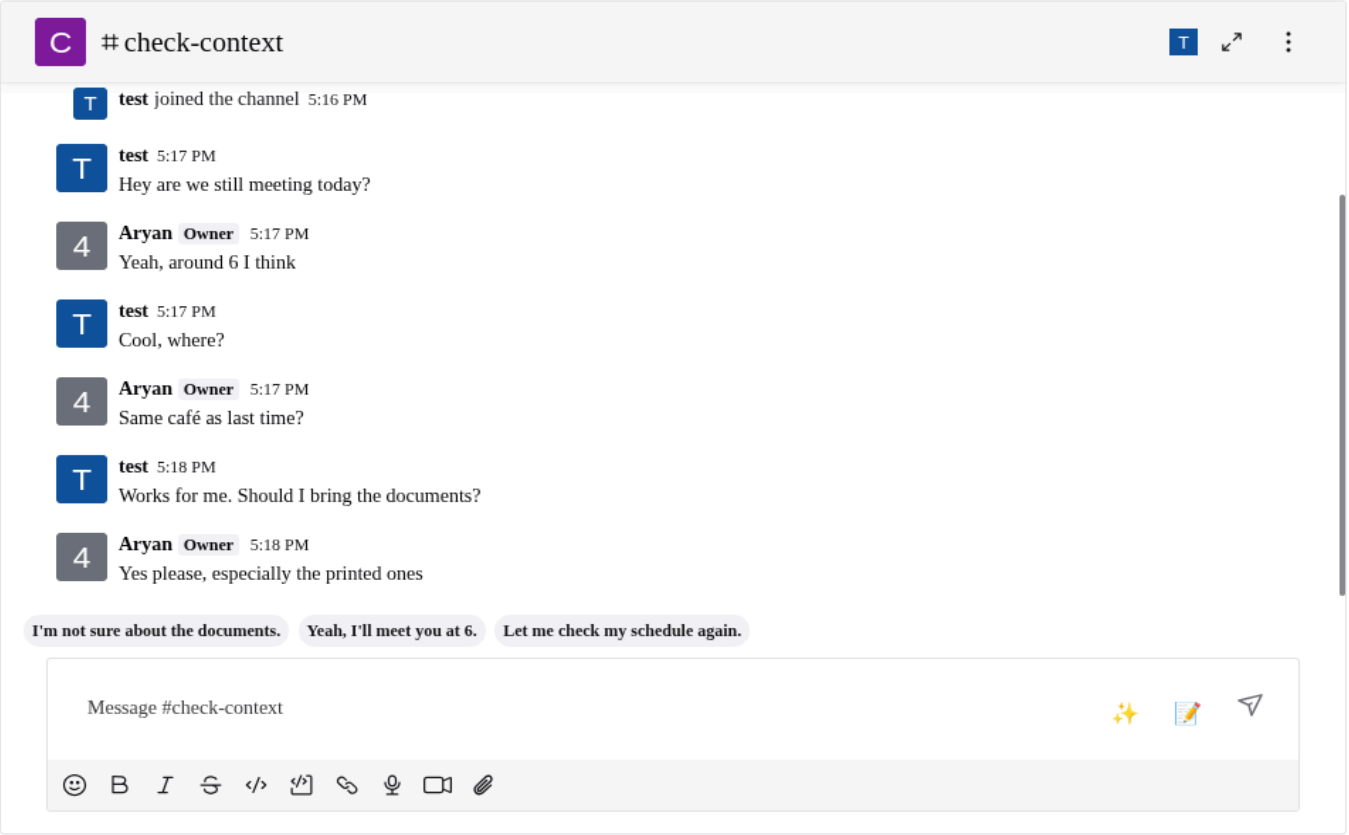


Fig. 12: Ai Suggestions demo using Ollama Adapter

```
const handleSummarize = async () => {
  if (!aiAdapter?.summarize || isSummarizing) return;
  setIsSummarizing(true);
  try {
    const { messages } = useMessageStore.getState();
    const result = await aiAdapter.summarize(messages);
    setSummary(result);
    setShowSummary(true);
  } catch (e) {
    console.error('[AI Adapter] summarize failed:', e);
  } finally {
    setIsSummarizing(false);
  }
};
```

Fig. 13: Ai Summary Hook inside chatinput

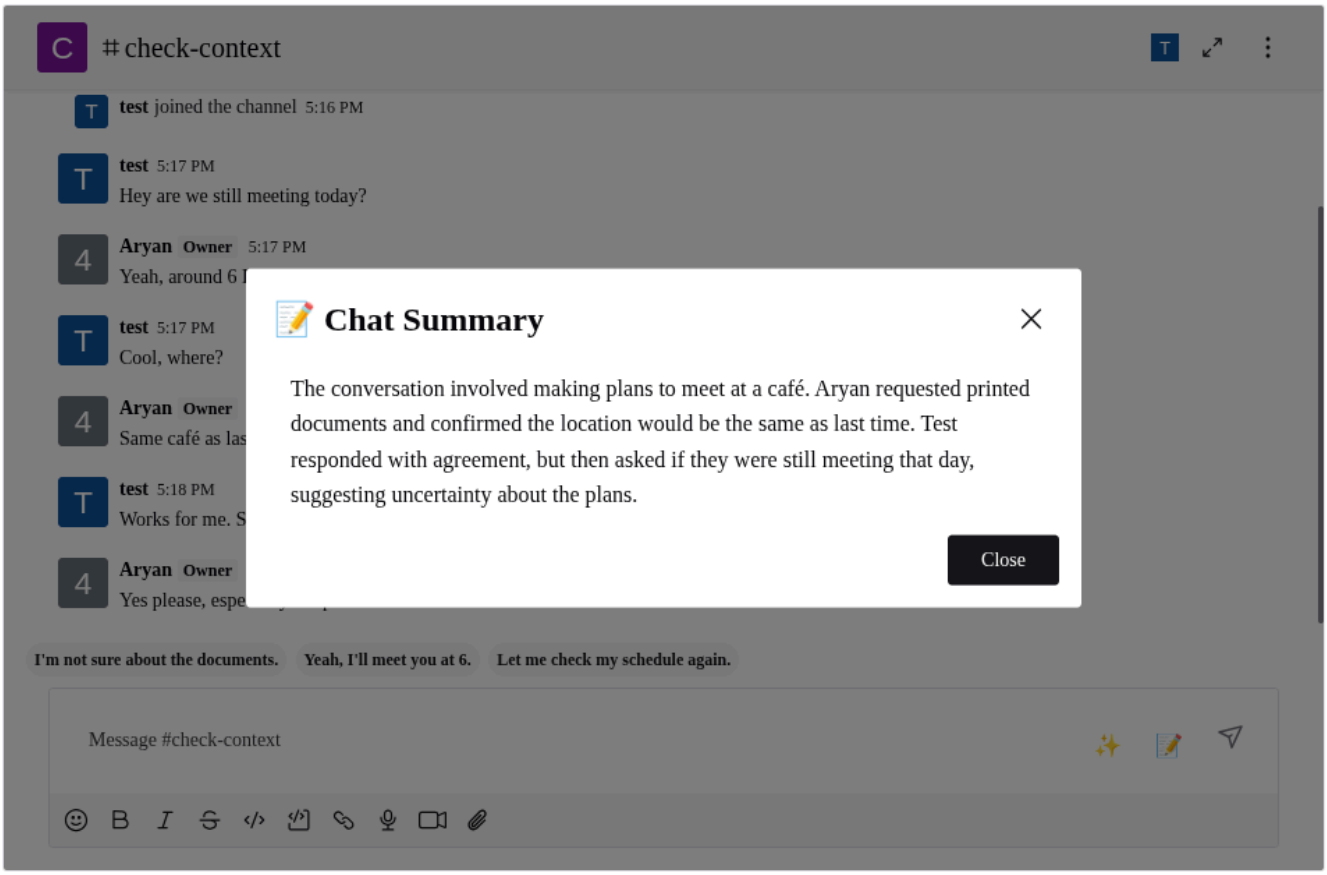


Fig. 14: Ai Summary demo using Ollama Adapter

4. Improving mobile responsiveness through reusable UI-layer fixes

Mobile responsiveness issues should be fixed at the shared component and interaction layer rather than with isolated one-off patches. My plan is to audit the widget at 375px and similar small-screen breakpoints, then fix the most visible usability problems by improving shared components in @embeddedchat/ui-elements and adapting mobile behavior in @embeddedchat/react.

Proposed architecture

The mobile work spans two layers:

- **@embeddedchat/ui-elements**
  Shared primitives such as menus, popups, modals, and buttons should adapt properly to small screens.
- **@embeddedchat/react**
  Feature-level views such as the emoji picker, chat input, attachment flows, and room actions will use those improved mobile-friendly primitives.

Implementation strategy

1. Audit the widget at common phone widths.
2. Identify UI primitives causing overflow or cramped behavior.
3. Shift unsuitable desktop popups into bottom-sheet or mobile overlay patterns.
4. Revalidate message composition, room actions, and modal flows.
5. Add Storybook coverage for mobile states.

```
if (isMobileBottomSheet) {
  return (
    <ReactPortal wrapperId={wrapperId}>
      <Box css={styles.mobileSheetBackdrop} onClick={onClose} />
      <Box
        css={styles.mobileSheet}
        onClick={(event) => event.stopPropagation()}
      >
        <Box css={[styles.emojiPicker, styles.mobileEmojiPicker]}>
          <EmojiPicker
            height="100%"
            width="100%"
            onEmojiClick={handleEmojiClick}
            previewConfig={previewConfig}
            autoFocusSearch={false}
            searchDisabled={false}
            emojiStyle="facebook"
            lazyLoadEmojis
          />
        </Box>
      </Box>
    </ReactPortal>
  );
}
```

Fig. 15: Mobile bottom-sheet behavior for the emoji picker

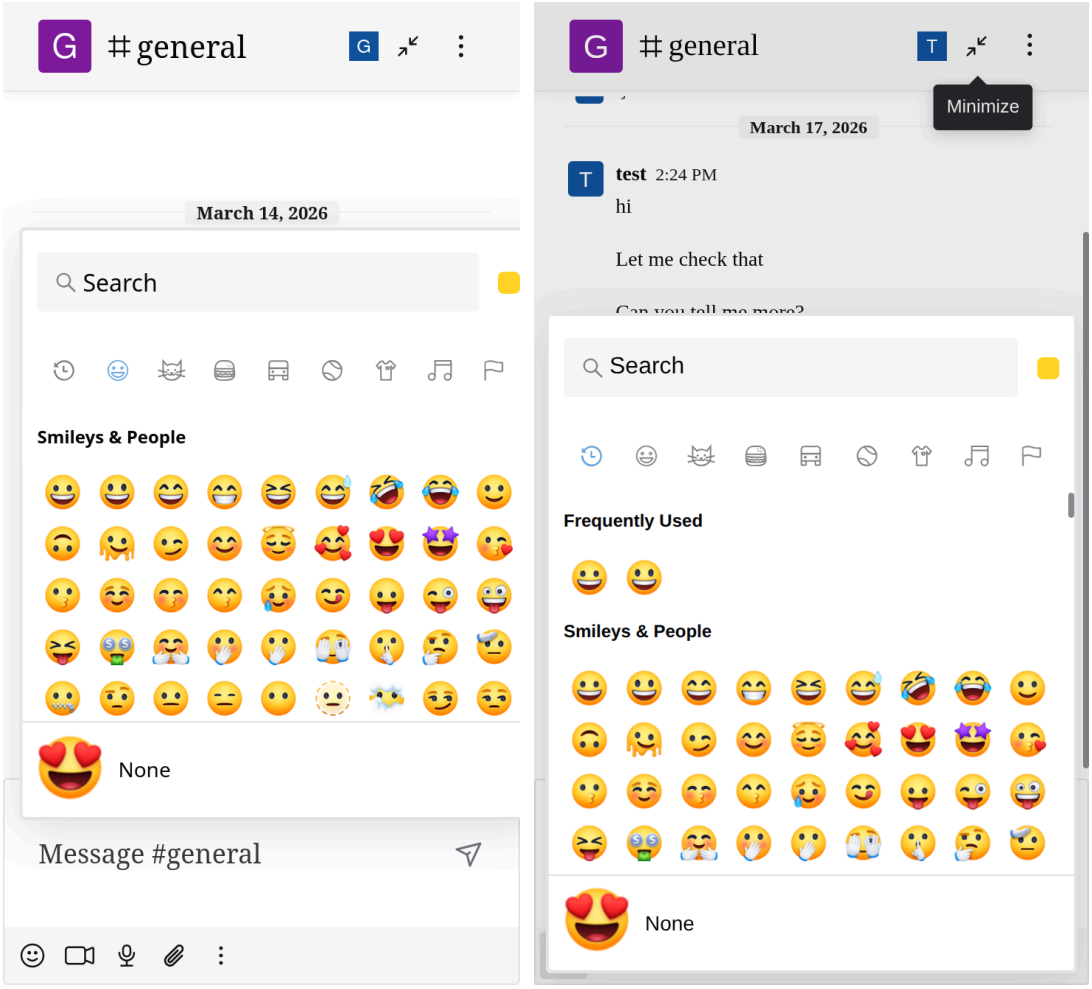


Fig. 16: Current behavior(left) vs Pitched Bottom Sheet behavior(right) Similar to RC

```
const optionJsx = (
  <>
    {triggerButton}
    {isOpen && isMobile ? (
      <>
        <Box css={styles.backdrop} onClick={() => setOpen(false)} />
        <Box
          css={[
            styles.sheet,
            css`
              box-shadow: ${theme.shadows[2]};
            `,
          ]}
          className={appendClassNames('ec-menu ec-menu-mobile', classNames)}
          style={styleOverrides}
          onClick={(e) => e.stopPropagation()}
        >
          {menuItems}
        </Box>
      </>
    ) : null}
    {isOpen && !isMobile ? (
      <Box
        css={[
          styles.container,
          css`
            box-shadow: ${theme.shadows[2]};
          `,
        ]}
        className={appendClassNames('ec-menu', classNames)}
        style={finalStyle}
      >
        {menuItems}
      </Box>
    ) : null}
```

Fig. 17: Mobile Sheet Implementation for Kebab Menu

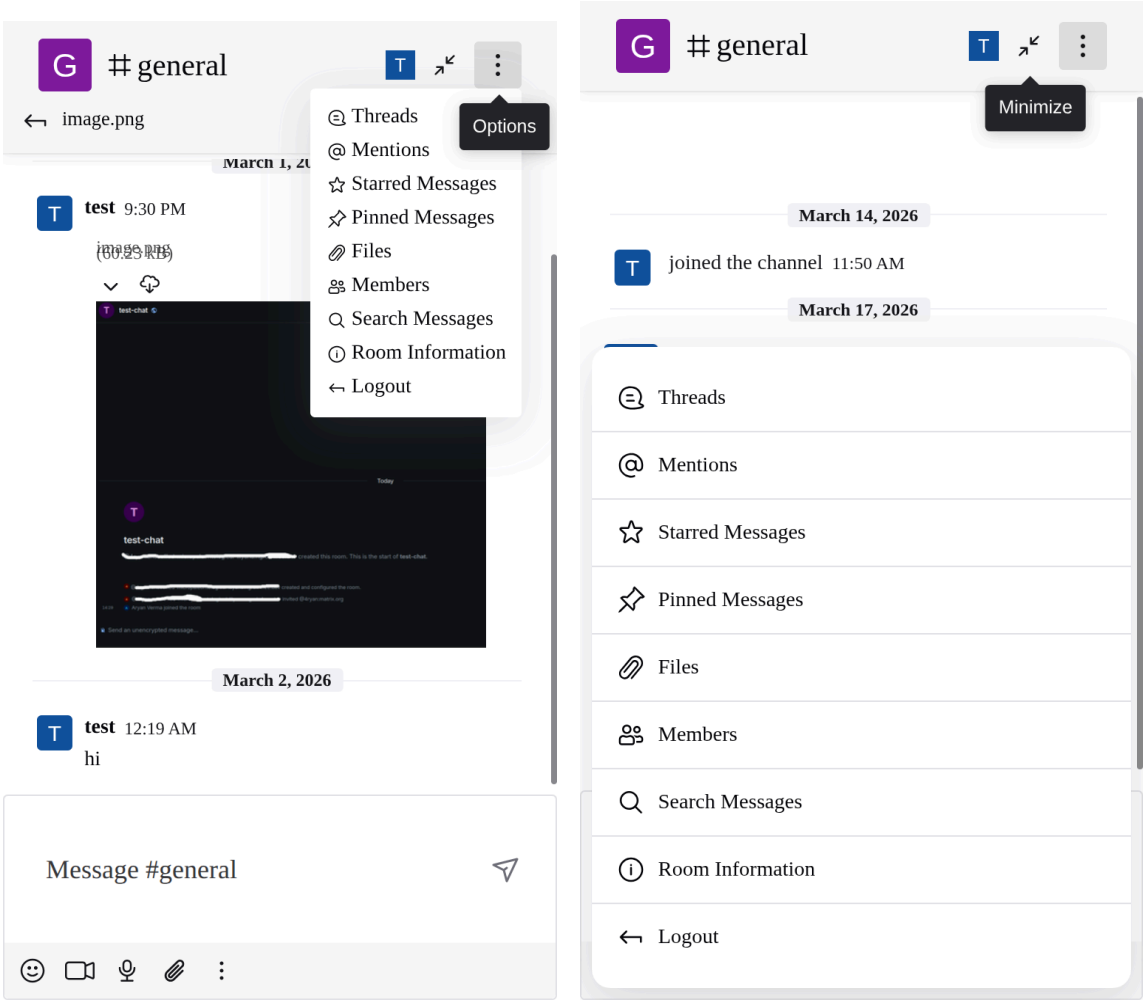


Fig. 18: Current behavior(left) vs Pitched Bottom Sheet behavior(right) Similar to RC

5. Bringing EmbeddedChat closer to WCAG 2.1 AA compliance

Accessibility improvements should be approached as a system-wide enhancement rather than a collection of isolated fixes. My plan is to strengthen semantic structure, keyboard navigation, focus handling, and dynamic announcements across the shared UI layer and the EmbeddedChat views built on top of it.

Proposed architecture

- **@embeddedchat/ui-elements**

  Shared controls should provide semantic and keyboard-friendly foundations.

- **@embeddedchat/react**

  Higher-level flows such as mention suggestions, command lists, input actions, and room navigation should use those primitives correctly and expose better accessibility behavior.

Implementation strategy

1. Audit the current widget for non-semantic interactive elements and missing keyboard support.
2. Replace weak interaction patterns with semantic ones.
3. Improve focus handling and navigation behavior.
4. Add ARIA roles, labels, and live-region support where needed.
5. Add automated accessibility checks to CI.

```
return (
  <Box
    css={[
      isMobile ? styles.itemMobile : styles.item,
      disabled && styles.disabled,
    ]}
    className={appendClassNames('ec-menu-item', classNames)}
    style={styleOverrides}
    onClick={!disabled && action}
  >
    <Icon name={icon} size={isMobile ? '1.25rem' : '1em'} />
    {label}
  </Box>
);
```

Fig. 19: Non-semantic clickable menu item in shared UI

```
const onKeyDown = (e) => {
  switch (true) {
    case e.ctrlKey && e.code === 'KeyI': {
      e.preventDefault();
      formatSelection(messageRef, '_{{text}}_');
      break;
    }
    case e.ctrlKey && e.code === 'KeyB': {
      e.preventDefault();
      formatSelection(messageRef, '*{{text}}*');
      break;
    }
    case (e.ctrlKey || e.metaKey || e.shiftKey) && e.code === 'Enter':
      e.preventDefault();
      handleNewLine(e);
      break;
    case e.code === 'Escape':
      if (editMessage.msg || editMessage.attachments) {
        e.preventDefault();
        messageRef.current.value = '';
        setDisableButton(true);
        setEditMessage({});
      }
      break;

    case e.code === 'Enter':
      e.preventDefault();
      if (!showCommandList && !showMembersList) {
        sendTypingStop();
        sendMessage();
      }
      break;
```

Fig. 20:Existing keyboard behavior in chat input

```yaml
.github > workflows > playwright.yml
name: Playwright Tests

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main, develop]

jobs:
  test:
    timeout-minutes: 60
    runs-on: ubuntu-22.04
    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Setup Node.js environment
        uses: actions/setup-node@v4
        with:
          node-version: 16.19.0

      - name: Cache dependencies
        uses: actions/cache@v4
        with:
          path: ~/.yarn
          key: ${{ runner.os }}-yarn-${{ hashFiles('**/yarn.lock') }}
          restore-keys: |
            ${{ runner.os }}-yarn-
```

Fig. 21:Existing Playwright baseline to extend with accessibility checks

6. Documentation and validation

Since this project spans compatibility, federation, AI integration, mobile usability, and accessibility, documentation will be a necessary part of implementation. I will maintain setup documentation, usage examples, and validation material for each area so that maintainers and future contributors can understand the final architecture without unnecessary effort.

I also plan to use Storybook, live POCs, and automated tests to keep each improvement demonstrable and reviewable throughout the project rather than only at the end.

☺ **Fun-fact:** Over half of work time is taken up searching for information.

DETAILED WORK PLAN

My proposed timeline for the project is as follows. This plan assumes the standard 12-week GSoC 2026 coding period for a 175-hour project.

Before May 01

- I will continue contributing to EmbeddedChat and stay active in discussions with the Rocket.Chat community and mentors.
- I will review Devansh Kansagra's PR #1009 in detail and document exactly which parts of Goal 1 are already covered and which parts still remain.
- I will clean up my existing working branches and organize my POCs for Matrix federation, AI adapter integration, mobile fixes, and accessibility observations so they can be later reused during implementation.
- I will prepare notes for each improvement area, especially around Rocket.Chat compatibility, Matrix federation architecture, and AI adapter design, so that the coding period starts with a clear technical plan instead of exploratory work.

Community Bonding Period Begins!

May 01- May 24

- With the help of my mentors, I will finalize the project scope, expected milestones, and weekly communication process.
- I will confirm the precise architecture for each improvement area, especially:
  1. Goal 1 modernization on top of PR #1009
  2. Matrix federation architecture and client/server responsibility split
  3. AI adapter contract and package boundaries
  4. Mobile responsiveness audit and component-level fix strategy
  5. Accessibility audit and automation strategy
- Take inspiration from existing code and will discuss with my mentor regarding how implementation can be done in the most efficient ways.
- I will also discuss edge cases with mentors early, especially around federation behavior, remaining DDP migration gaps, and React/Node upgrade risks.
- I will participate in discussions with the community to know the future plans of Rocket.Chat organization.

Coding officially begins!

Week 1 [May 25 – May 31]

- Reproduce the current compatibility issues against a current Rocket.Chat setup and create a baseline checklist for broken and partially migrated flows.
- Start Improvement 1 by extending the work already done in PR #1009 and auditing remaining legacy or fallback paths in @embeddedchat/api and @embeddedchat/auth.
- Validate current build, Storybook, and end-to-end status before making major modernization changes.

Week 2 [June 1 – June 7]

- Continue Goal 1 by completing the remaining API/auth migration work.
- Replace outdated or removed Rocket.Chat endpoint usage where needed.
- Fix integration issues that appear in @embeddedchat/react because of API/auth changes.
- Validate the main chat flow end to end.

Week 3 [June 8 – June 14]

- Upgrade the monorepo toward a unified React version 19 across core packages.
- Upgrade the repo from the current Node 16 baseline to the latest stable Node.js 22.
- Fix Storybook, demos, and build/test issues caused by the React/Node upgrade.
- Keep this week focused on stabilization.

Week 4 [June 15 – June 21]

- Start Matrix federation work.
- Finalize the federation-aware client architecture in EmbeddedChat.
- Integrate the base federation mode flow in the React package.
- Begin wiring federation UI states and room handling.

Week 5 [June 22 – June 28]

- Continue Matrix federation integration and validate it against my existing POC setup.
- Improve the UI for federated rooms, badges, and unsupported states.
- Keep documentation notes as I go so setup is easier to explain later.

Week 6 [June 29 – July 5]

- Start the AI adapter layer
- Stabilize the package structure and reference adapters.
- Integrate adapter injection into EmbeddedChat through props/context.

Midterm Evaluation

July 6 - July 10

- Work on drafting a report for midterm evaluations.
- Submit the midterm report.
- Gather mentor feedback and adjust the second half if any area needs prioritization.

Coding continues!

Week 7 [July 11 – July 17]

- Continue the AI adapter work.
- Finish user-facing AI integration such as suggestions and summarization.
- Add examples and documentation for using the adapter package.

Week 8 [July 18 – July 24]

- Start the mobile responsiveness phase.
- Fix the most important issues first, such as emoji picker behavior and menu overflow
- Push reusable fixes into shared UI components wherever possible.

Week 9 [July 25 – July 31]

- Continue mobile improvements and polish the remaining high-impact issues.
- Start accessibility improvements with a focused audit of interactive elements.
- Prioritize keyboard navigation, semantic controls, and the most visible accessibility gaps.

Week 10 [August 1 – August 7]

- Continue accessibility work.
- Improve semantics, focus handling, keyboard interaction, and dynamic announcements where needed.
- Add or extend automated checks for accessibility-related regressions if feasible within scope.

Week 11 [August 8 – August 16]

- Reserve this week mainly for integration testing and bug fixing.
- Re-test modernization, federation, AI adapter, mobile, and accessibility changes together.
- Finish documentation for setup, usage, and limitations.
- Use this week as the main buffer for anything that slips slightly earlier.

Final Evaluation

August 17- August 24

- Submit final work product and evaluation materials.
- Finalize documentation and clean up any remaining review comments.
- Prepare summary of deliverables, known limitations, and suggested follow-up work.
-

Post GSoC Goals

- Continue contributing to EmbeddedChat after GSoC.
- Keep improving the Matrix federation and AI adapter work based on mentor and feedback.
- Implement the functionality mentioned in Extras after approval from the mentor.

RELATED WORK

I have already been contributing since December closely related to the scope of this proposal across both RocketChat/EmbeddedChat and RocketChat/Rocket.Chat. These contributions are directly aligned with the main areas I want to work on during GSoC: Rocket.Chat compatibility, Matrix federation, mobile usability, accessibility, and extensibility. Through this work, I have become familiar with EmbeddedChat's monorepo structure, the API/auth flow, Storybook-based testing, shared UI components, and the Rocket.Chat server-side constraints that affect federation and compatibility.

Recent work especially relevant to this proposal includes the following:

Rocket.Chat modernization and compatibility

1. **[OPEN]** EmbeddedChat #1171 : fix: disconnect WebSocket on logout to prevent ghost sessions
2. **[OPEN]** EmbeddedChat #1173 : fix(api): migrate file upload from removed rooms.upload to rooms.media endpoint
3. **[OPEN]** EmbeddedChat #1174 : fix(api): replace deprecated method.call endpoints with REST equivalents
4. **[OPEN]** EmbeddedChat #1175 : fix(api): remove deprecated query/fields params, filter threads client-side
5. **[OPEN]** EmbeddedChat #1188 : fix(api): add missing await on response.json() in getUserStatus, userInfo, and userData
6. **[OPEN]** EmbeddedChat #1187 : chore: update Node to v22 LTS and replace console.log with console.error in catch blocks

These PRs are especially relevant to Goal 1 because they show that I am already working on the same compatibility surface this proposal targets: removed endpoints, response handling, WebSocket lifecycle, and runtime modernization.

Matrix federation and Rocket.Chat server-side understanding

1. [DRAFT] EmbeddedChat #1030 : feat: Matrix Integration for EmbeddedChat
2. **[OPEN]** Rocket.Chat #38669 : test(federation): add e2e test for self-join in federated rooms
3. **[OPEN]** Rocket.Chat #39427 : fix(federation): register local users on Matrix when self-joining federated rooms

This part shows that my federation work is not limited to a UI-only demo. I have also worked on reproducing and fixing federation behavior at the Rocket.Chat server level, which gives me a much clearer view of where EmbeddedChat integration ends and where Rocket.Chat federation internals begin.

Mobile usability and UI refinement

1. **[OPEN]** EmbeddedChat #1195 : fix(ui): resolve attachment metadata text overlap on mobile viewports
2. **[OPEN]** EmbeddedChat #1220 : fix(menu): add mobile bottom sheet for kebab
3. **[OPEN]** EmbeddedChat #1223 : feat: improve mobile emoji picker experience

These PRs are directly connected to my proposed mobile-responsiveness work. They reflect ongoing work on real small-screen usability problems rather than speculative improvements.

Accessibility and interaction quality

1. **[OPEN]** EmbeddedChat #1176 : a11y: add aria-labels to all icon-only buttons and interactive elements
2. **[OPEN]** EmbeddedChat #1181 : a11y: add semantic roles, aria-live, and keyboard navigation to main containers

These PRs show that I have already started addressing accessibility from both the semantic and interaction sides, which is directly relevant to the WCAG-focused part of my proposal.

In addition to these PRs, I already have working POCs for Matrix federation, AI adapter integration, and mobile UI fixes. Together, these contributions show that I am not approaching this project from scratch. I have already been working on the same parts of the codebase and ecosystem that this proposal aims to improve, which gives me a practical foundation for executing the proposed work during GSoC.

You can also verify working of my Matrix federation and Ai Adapter demo by visiting chat.aryanverma.dev by using credentials :

Username: gsoc
Password: Embeddedchat@2026

RELEVANT EXPERIENCES

This is my first time participating in GSoC, and I came to open source contributions relatively recently. But I have been a user of open-source software for as long as I can remember: browsing on Firefox, daily driving Pop OS as Operating System, watching films on VLC, writing code in VS Code. Over time, that familiarity turned into a genuine desire to contribute to projects I use and care about.

I started contributing to Rocket.Chat after coming across EmbeddedChat issue #1029, opened by Zishan Ahmad. I jumped in head first without any prior understanding of Matrix federation or Rocket.Chat homeserver internals which turned out to be both the mistake and the education.

My first attempt went in the wrong direction entirely. I built an implementation around direct Matrix-side integration using the Matrix JS-SDK, which became PR #1030. When Zishan reviewed it, he redirected me: the goal was not to log in with Matrix credentials directly, but to work through Rocket.Chat's native federation layer instead. I won't pretend I didn't panic. I had spent weeks building what I thought was the right solution, and in one review comment it became clear I had misunderstood the entire architecture. But Zishan was calm and encouraging, he told me to explore further rather than abandon the work.

What followed were weeks of reading undocumented source code. The RC homeserver repository has no client documentation. I read the entire codebase to understand what it was doing, reached out to core engineers, and explored every option: Matrix SDK, Matrix bridge, the RC homeserver, and finally RC's native federation, which was still in alpha when I found it. I watched it move from alpha to beta. It still doesn't support public Matrix room discovery or end to end encryption, but the team is actively working toward both and I have been watching that progress closely enough to know exactly where the gaps are.

This process of exploring the wrong path, receiving mentor feedback, understanding why it was wrong, and rebuilding on a stronger foundation taught me more about software architecture and system boundaries than my four years of coursework. It is also why my federation implementation today is stronger because it comes from iterations and months of reading undocumented code.

Beyond my individual contributions, I have also been actively involved in the Rocket.Chat community. I attended the RC Apps AI Workshop recently in which I was tasked to build a mention bot with slash commands, ephemeral messaging, and an external logger from scratch. I completed the challenge and honestly I enjoyed it more than I expected.The mentor sessions were genuinely fun and gave me a much clearer picture of how RC apps fit into the broader ecosystem. It was also nice to interact with the mentors outside of PR review comments for once.

Since then I have contributed across EmbeddedChat and the main Rocket.Chat repository. A detailed list of all my PRs and issues is provided below:

ISSUES:

Rocket.Chat/Rocket.Chat

- Issues (Total: 1)

Rocket.Chat/EmbeddedChat

- Issues (Total: 6)

PULL REQUESTS:

Total: 16 **Under Review: 16**

Open PRs

1. [OPEN] EmbeddedChat #1171 : fix: disconnect WebSocket on logout to prevent ghost sessions
2. [OPEN] EmbeddedChat #1173 : fix(api): migrate file upload from removed rooms.upload to rooms.media endpoint
3. [OPEN] EmbeddedChat #1174 : fix(api): replace deprecated method.call endpoints with REST equivalents
4. [OPEN] EmbeddedChat #1175 : fix(api): remove deprecated query/fields params, filter threads client-side
5. [OPEN] EmbeddedChat #1188 : fix(api): add missing await on response.json() in getUserStatus, userInfo, and userData
6. [OPEN] EmbeddedChat #1187 : chore: update Node to v22 LTS and replace console.log with console.error in catch blocks
7. [DRAFT] EmbeddedChat #1030 : feat: Matrix Integration for EmbeddedChat
8. [OPEN] Rocket.Chat #38669 : test(federation): add e2e test for self-join in federated rooms
9. [OPEN] Rocket.Chat #39427 : fix(federation): register local users on Matrix when self-joining federated rooms
10. [OPEN] EmbeddedChat #1195 : fix(ui): resolve attachment metadata text overlap on mobile viewports
11. [OPEN] EmbeddedChat #1220 : fix(menu): add mobile bottom sheet for kebab
12. [OPEN] EmbeddedChat #1223 : feat: improve mobile emoji picker experience
13. [OPEN] EmbeddedChat #1176 : a11y: add aria-labels to all icon-only buttons and interactive elements
14. [OPEN] EmbeddedChat #1181 : a11y: add semantic roles, aria-live, and keyboard navigation to main containers
15. [OPEN] EmbeddedChat #1231 :critical fix: Image attachments return 403 added Rc auth tokens to image URLs
16. [OPEN] EmbeddedChat #1234 : fix: resolve 400 Bad Request and restore message loading for public channels

SUITABILITY

Why are you the right person to work on this project?

I am the right person to work on this project because I have spent the last six months contributing deeply to both EmbeddedChat and the main Rocket.Chat repository, and I have working POCs for every goal in this proposal. I understand the codebase well enough that I will not spend GSoC time getting oriented. I will spend it building.

I did not plan to become the person who reads undocumented Rocket.Chat Enterprise Edition source code at 2am. But that is what happened, and looking back, I would not

change any of it.

I started with PR #1030 thinking I was building a simple Matrix mode toggle. I had no prior understanding of Matrix federation, no familiarity with Rocket.Chat's homeserver internals, and frankly no idea what I was getting into. A few weeks later I was staring at Rocket.Chat's Enterprise code, trying to understand why a single inviter check was silently breaking federation for every user who tried to self-join a federated room. I filed issue #38239. Then ggazzo asked me to write an E2E test to reproduce it. I did. Then I wrote the fix itself. Then I deployed a patched RC instance, connected my EmbeddedChat fork, and watched two users on different Matrix servers send messages to each other through code I had written.

That is the moment I knew I wanted to see this project through.

Since then I have contributed across both repositories: API migration fixes for RC 8.x breaking changes, accessibility improvements, mobile viewport fixes, an AI adapter POC with live suggestions and thread summarization, and talked to Devansh Kansagra about his DDP migration PR. I have spoken with Zishan directly about architecture decisions and read more undocumented source code than I care to admit.

I am comfortable in TypeScript, React, Node.js, and the Matrix protocol. I know EmbeddedChat's codebase well enough.

I also care about the community beyond my own PRs. I have helped fellow contributors navigate the project in the public channel, reviewed others' work, and engaged genuinely rather than just hunting issues for the leaderboard.

The Rocket.Chat community has taught me more in six months than I learned in four years of coursework. I want to give that back, and I plan to keep contributing long after GSoC ends.

TIME AVAILABILITY

How much time do you expect to dedicate to this project?
I am a recent graduate with no ongoing academic commitments, employment, or other obligations during the GSoC period. I can dedicate 40 to 45 hours per week to this project without any competing demands on my time.

Please list jobs, summer classes, and/or vacations that you will need to work around.

I have not enrolled in any summer classes and I have no plans to travel or take vacation during the GSoC period. I do not have a job or internship lined up. GSoC will be my primary focus from day one. I will be available for calls from 10am to 10pm IST on both weekdays and weekends.